

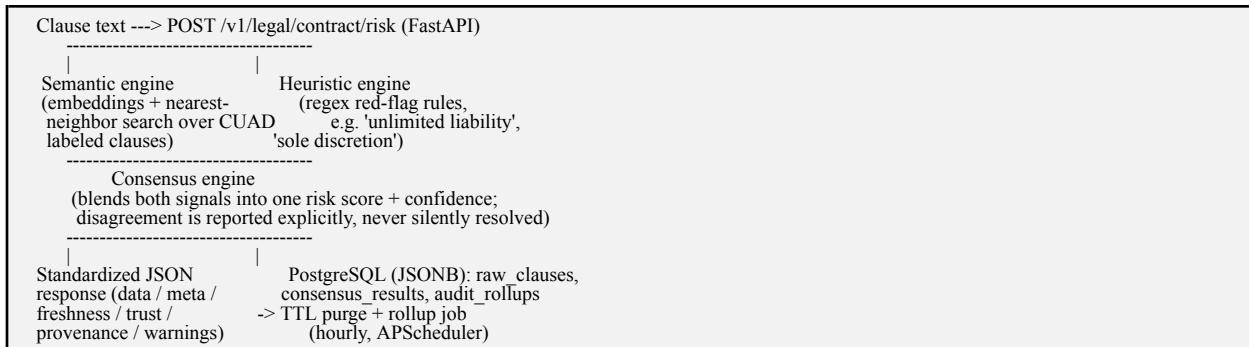
Redline: Contract Clause Risk Consensus API

AI4012 — Assignment #1 | Task: Legal / Contract Clause Risk

Part 1 — System Architecture

Redline evaluates the risk of a single contract clause by combining two independent signals into one consensus result, rather than trusting a single model or a single rule set. This satisfies the assignment's multi-source consensus requirement in a domain (legal text) where a single ground-truth API does not exist.

Data flow



The two independent sources

- **Semantic engine:** a sentence-transformer embeds the incoming clause and compares it against a pre-built index of the CUAD dataset (Contract Understanding Atticus Dataset) — real, expert-labeled contract clauses spanning categories such as Uncapped Liability, Non-Compete, and Auto-Renewal. The closest match's category and similarity score become the first signal.
- **Heuristic engine:** an independent rule-based scanner checks the clause against a curated list of known red-flag legal phrases (e.g. 'sole discretion', 'unlimited liability', 'perpetual', 'without cause'). This signal does not depend on the embedding model at all, so it keeps working even if the semantic engine is degraded.

Consensus logic

The two signals are blended into a single risk score and a confidence score. When both signals agree that a clause is (or is not) risky, confidence is high and the result is marked verified. When they disagree, Redline does not silently pick a winner: it lowers the confidence score, marks verified as false, and attaches an explicit warning recommending manual review. This directly implements the assignment's 'no silent fallbacks' requirement — a disagreement is a visible, auditable state, not a hidden default.

Storage & lifecycle (PostgreSQL + JSONB)

- **raw_clauses** — every incoming request, stored as JSONB with a timestamp.
- **consensus_results** — the computed semantic score, heuristic flags, blended risk score, and confidence, linked to its raw request.
- **audit_rollups** — daily aggregate counts written by an hourly background job before older raw request rows are purged past their TTL, satisfying the automated lifecycle-management requirement.

Serving layer & SLA

A FastAPI endpoint (POST /v1/legal/contract/risk) returns the exact standardized envelope specified in the assignment. Since the semantic index is pre-built at startup rather than queried live, the hot path has no external calls — keeping p95 latency well under the 200ms SLA and making failure explicit (a missing index returns a 503, never a degraded, mislabeled result). Automated tests cover availability, p95 latency, freshness-SLA consistency, consensus correctness, and failover.

Part 2 — Use of AI

AI assistance (Claude) was used in this project for planning and code generation. No AI tool was used to search for or copy an existing solution to this assignment; all AI usage was for implementation support on a design and task selection I made myself.

How AI was used

- Task selection: discussed trade-offs between candidate tasks from the provided data catalog (e.g. drug interaction lookup, candidate/job matching, contract clause risk) weighing implementation risk against how well each task could showcase ML/DS skills, given the assignment deadline.
- Architecture design: used AI to think through how to structure a genuinely independent two-signal consensus system (semantic similarity vs. rule-based heuristics) rather than two near-duplicate API calls, and how to keep the required TTL/lifecycle logic meaningful even though this task's underlying data (CUAD corpus, rule set) is static rather than continuously live.
- Code structuring: AI generated the initial FastAPI application structure, Pydantic schemas matching the required response envelope, the heuristic rule engine, the embedding/consensus logic, the PostgreSQL JSONB schema and TTL purge job, and the pytest suite (availability, latency, freshness, failover, consensus-correctness tests). All generated code was reviewed, run, and verified locally before being treated as part of the submission.
- Frontend demo: AI generated a lightweight HTML/JS demo page that calls the API and visually surfaces both signals plus a disagreement warning, used to sanity-check end-to-end behavior.

Representative prompts used

- "Sketch the architecture for a contract clause risk API that combines an embeddings-based similarity check against CUAD with a rule-based heuristic check, reconciled into one consensus score."
- "Write the FastAPI endpoint and Pydantic schemas so the response matches this exact JSON envelope [assignment spec], including provenance, freshness, and trust fields."
- "Write pytest tests that verify p95 latency under 200ms, freshness SLA field consistency, and that the API fails explicitly (not silently) when the semantic engine is unavailable."
- "Design a PostgreSQL JSONB schema for raw requests, consensus results, and TTL rollups, with appropriate indexes."

What I did myself

I selected the final task and rejected several AI-suggested alternatives based on my own risk/deadline judgment, configured and ran the project locally (Docker Postgres, Python environment, CUAD download), adjusted the CUAD column-name handling to match the actual dataset release, verified all tests passed on my machine, and wrote this report.